Classification with probabilities

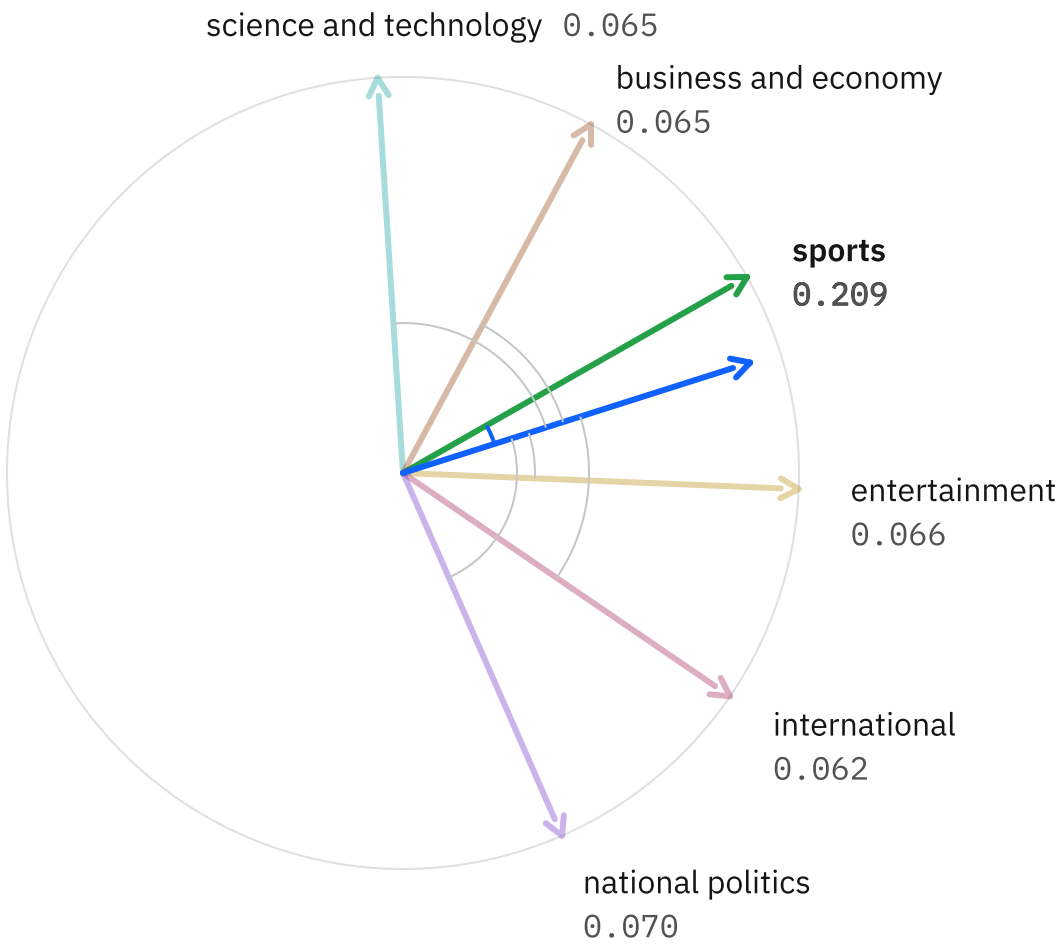
Class 2 of 24: Natural Language Processing

CSE 4811 / DS 4213 (Fall 2026)

Muhammad Anwarul Azim
Lecturer, Dept. of CSE
United International University

Recap

A new article: its cosine with each average. The topic with the largest cosine similarity is selected.



a new article

প্রথম ও শেষ কথা জয়: মাশরাফি
বিশ্বকাপে নিজেদের পঞ্চম ম্যাচে সোমবার
...

nearest average: **sports (right)**

The six cosines are computed by the class 1 model; the angles are a sketch.

How good was our classification?

Consider one topic: Sports

518 news classified as sports



497 right — 21 wrong

The 21 wrong were really

national politics	3
international	4
business and economy	2
entertainment	6
science and technology	6

666 texts really sports



497 found — 169 missed

The 169 missed were called

national politics	15
international	27
business and economy	76
entertainment	49
science and technology	2

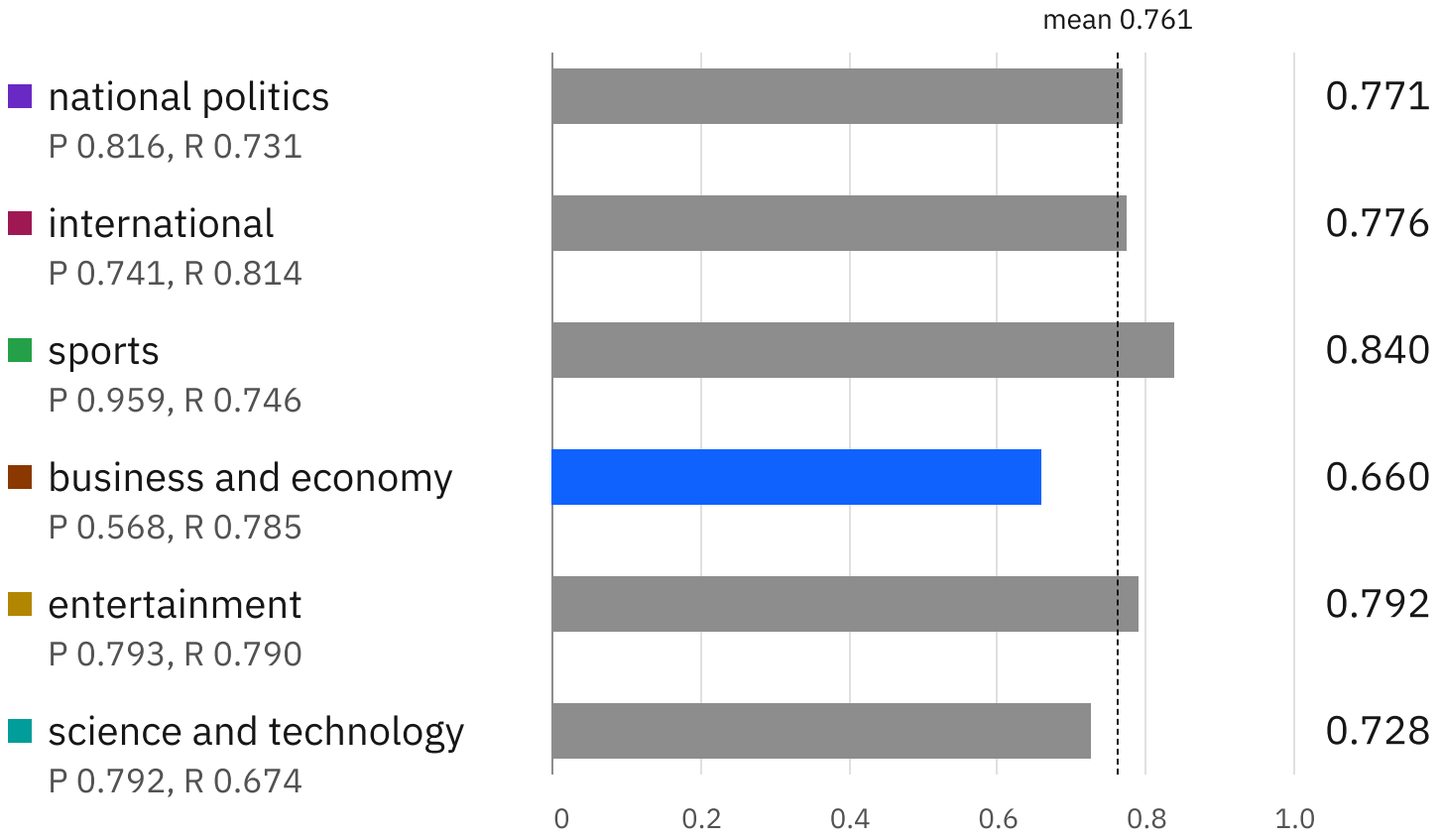
$$\text{Precision} = \frac{497}{518} = 0.959$$

$$\text{Recall} = \frac{497}{666} = 0.746$$

$$\text{F1} = \frac{2 \times 0.959 \times 0.746}{0.959 + 0.746} = 0.840$$

Why macro-F1?

F1 for each topic. Nearest topic average on TF-IDF, 4,000 dev texts.



0.761

macro-F1, the mean of the six

0.757

accuracy, the share of texts right. Our dev set is balanced, so accuracy and macro-F1 are close.

Macro-F1 gives every topic the same weight.

Our score

Dev set: 4,000 news articles that were not used to compute the six topic averages

0.761

macro-F1



973 of 4,000 wrong

Training set: the 40,000 news articles used to compute the six topic averages

0.782

macro-F1



8,848 of 40,000 wrong

It also misclassifies 22 % of the texts its averages were computed from.

Part 1: How to improve

What the nearest topic average cannot do, and what a learning model needs.

Two major limitations

1

It ignores word order

```
Bangladesh beat India
India beat Bangladesh
```

the same vector | `bangladesh 1` | `beat 1` | `india 1`

2

It can't learn

8,848 of 40,000 wrong.

Mistakes are normal. A learning system must learn from its mistakes.

To learn from mistakes, we need

a loss function
how wrong the model is

→

a way to adjust the model
so that the loss falls

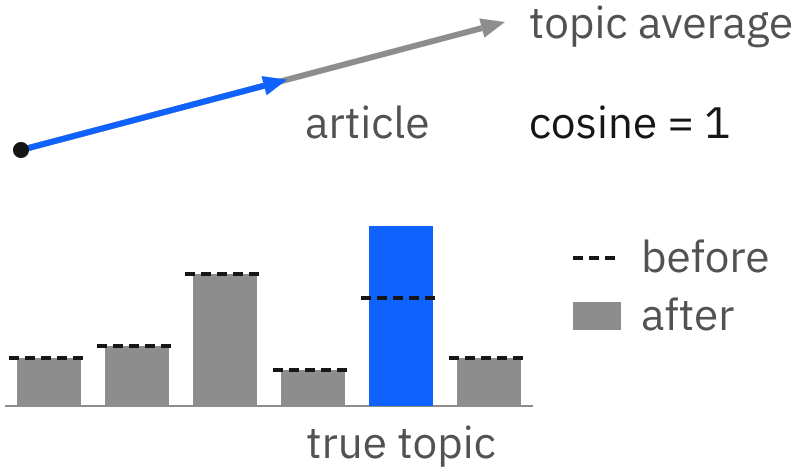
Today: the loss

Why cosine cannot be the loss

Suppose we train the average model to make the cosine larger.

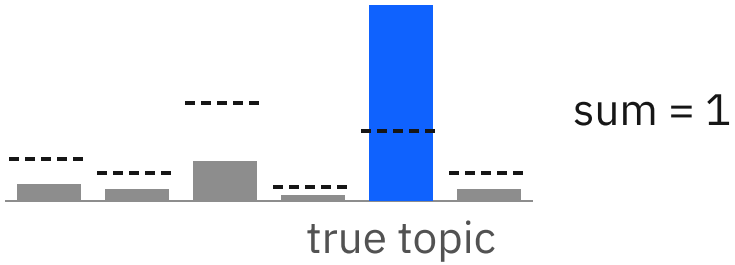
1. A cosine of 1 is not the goal
A cosine of 1 means the article has the same mix of words as its topic average. We never wanted that.

2. Cosine does not move away from the rivals
A learning model should move toward the true topic and away from the false topics. Cosine scores each topic on its own. A larger cosine for the true topic leaves the other cosines as they are.



We need probabilities
Each news item belongs to one topic. Our goal is to be fully confident of that topic: $p(\text{true topic} \mid \text{article}) = 1$.

The six probabilities add up to 1. When the true topic's probability rises, the probabilities of the other topics fall.



1. Probability → 2. Loss → 3. Adjustment

Part 2: Computing probabilities

From word counts to the probability of each topic.

Probability of a topic

Given a text, how probable is each topic?

$$p(k \mid \text{text})$$

---

k — A topic: one of the six topics of our data set. In the toy example, k is sports or entertainment.

- international
- national politics
- business and economy
- entertainment
- sports
- science and technology

---

text — A news article from our data set. We count its words and ignore their order, so we represent it as a bag of words: its words, each with its count.

`Star film wins` → `film` `star` `wins`

`Wins film star` → `film` `star` `wins` the same bag

$p(k \mid \text{text})$ answers this: after we read the words of the article, how probable is topic k?

Bayes' formula

$$p(k \mid \text{text}) = \frac{p(\text{text} \mid k)\ p(k)}{\sum_j p(\text{text} \mid j)\, p(j)}$$

$p(k)$ **Prior**

How common topic k is, before we read the article.

$\sum_j p(\text{text} \mid j)\, p(j)$ **Normalizer**

The same for every topic. It makes the results add up to 1.

$p(\text{text} \mid k)$ **Likelihood**

How probable this bag of words is for topic k.

$p(k \mid \text{text})$ **Posterior** our goal

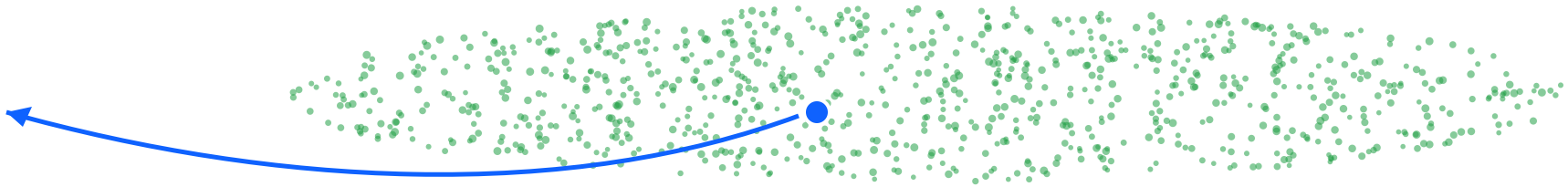
How probable topic k is after we read the article.

Our article

`Star film wins`

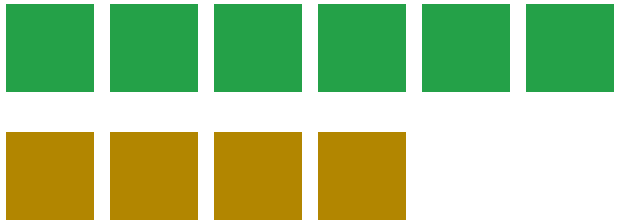
`film` `star` `wins`

A room with the bags of words of all sports articles



This chance is usually a very small number.

The prior



6 sports

4 entertainment

$$p(k) = \frac{n_k}{N}$$

n_k = articles in topic k

N = all training articles

- $p(\text{sports}) = 6/10 = 0.6$
- $p(\text{entertainment}) = 4/10 = 0.4$

C1 training set: 40,000 articles



international	national politics	business and economy	entertainment	sports	science and technology
0.170	0.170	0.170	0.170	0.164	0.155

In the C1 training set, the six topics are almost equally common.

The likelihood

Test headline `Star film wins`

Word counts	match	star	film	wins	song	Total
sports	0.40	0.20	0.05	0.30	0.05	1
entertainment	0.05	0.30	0.35	0.10	0.20	1

$$p(w \mid k) = \frac{\text{count}(w, k)}{\text{words in topic } k}$$

$$p(\text{text} \mid k) = \prod_i p(w_i \mid k)$$

- $p(\text{text} \mid \text{sports}) = 0.20 \times 0.05 \times 0.30 = 0.003$
- $p(\text{text} \mid \text{entertainment}) = 0.30 \times 0.35 \times 0.10 = 0.0105$

Note: with hundreds of words, this product becomes too small for the computer to store. We fix this later.

The posterior

Test headline `Star film wins`

Topic	Likelihood	Prior	Likelihood × prior	Posterior
sports	0.003	0.6	0.0018	0.0018 / 0.0060 = **0.30**
entertainment	0.0105	0.4	0.0042	0.0042 / 0.0060 = **0.70**
Sum		**1**	**0.0060**	**1**



The prior favored sports and the words favored entertainment. The posterior weighs both.

Classification

$$\hat{y} = \arg\max_k \; p(k \mid \text{text})$$

$$= \arg\max_k \; p(\text{text} \mid k)\, p(k)$$ the normalizer is the same for every topic

`Star film wins`

Topic	Likelihood × prior	Posterior
sports	0.0018	0.30
entertainment	**0.0042**	**0.70**

Prediction: **entertainment**

`Star wins match`

Topic	Likelihood × prior	Posterior
sports	**0.0144**	**0.96**
entertainment	0.0006	0.04

Prediction: **sports**

For Star wins match, the same rule predicts sports, with posterior 0.96.

The Naive Bayes classifier

Prior	$p(k) = \frac{n_k}{N}$
Word probabilities	$p(w \mid k) = \frac{\text{count}(w, k)}{\sum_v \text{count}(v, k)}$
Likelihood, the naive assumption	$p(\text{text} \mid k) = \prod_i p(w_i \mid k)$
Posterior, Bayes' formula	$p(k \mid \text{text}) = \frac{p(\text{text} \mid k)\, p(k)}{\sum_j p(\text{text} \mid j)\, p(j)}$
Prediction	$\hat{y} = \arg\max_k \; p(k \mid \text{text})$

Problem: for an article of hundreds of words, the product of probabilities underflows to 0.

Underflow

An article of 300 words, each with probability about 0.002

Words	Product	Sum of logs
1	2.0×10^{-3}	−6.2
2	4.0×10^{-6}	−12.4
3	8.0×10^{-9}	−18.6
⋮	⋮	⋮
119	6.7×10^{-322}	−739.5
120	**0**	−745.8
⋮	⋮	⋮
300	**0**	−1864.4

$$\log(ab) = \log a + \log b$$

$z_k = \log p(k) + \sum_i \log p(w_i \mid k)$ the score of topic k

Star film wins	z_k	$z_k - \max$	$e^{z_k - \max}$	÷ sum
sports	−6.32	−0.85	0.43	**0.30**
entertainment	−5.47	0	1	**0.70**

$$p(k \mid \text{text}) = \frac{e^{z_k}}{\sum_j e^{z_j}}$$ softmax

Solved: compute with sums of logs, and turn the scores into probabilities with softmax.

Naive Bayes with logs

Word probabilities, with 1 added	$p(w \mid k) = \dfrac{\text{count}(w,k)+1}{\sum_v \text{count}(v,k)+V}$
Score	$z_k = \log p(k) + \sum_i \log p(w_i \mid k)$
Prediction and probabilities	$\hat{y} = \arg\max_k z_k, \qquad p(k \mid \text{text}) = \text{softmax}(z)_k$

```
counts = np.vstack([np.asarray(X_train[y == k].sum(axis=0)).ravel()
                    for k in range(6)])                    # 6 x V word counts
counts = counts + 1                                        # add 1
log_prior = np.log(np.bincount(y) / len(y))                # log p(k)
log_p = np.log(counts / counts.sum(axis=1, keepdims=True))  # log p(w | k)
Z = log_prior + X_dev @ log_p.T                            # scores
pred = Z.argmax(axis=1)                                    # predictions
P = softmax(Z)                                             # probabilities
```

The same steps in Python, on the 40,000 training articles.

The full table

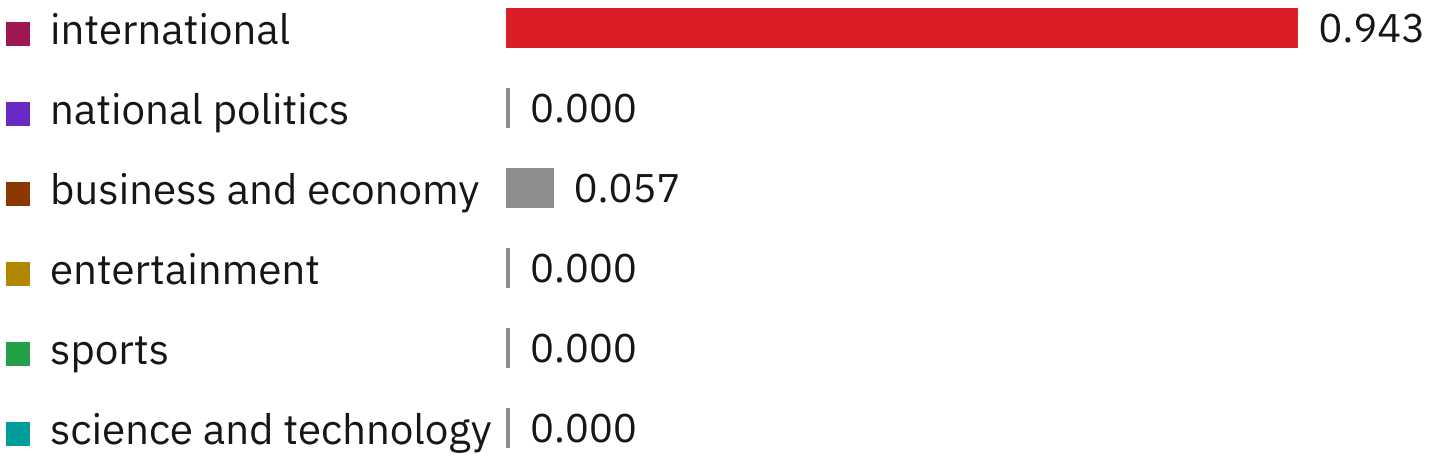
> A dev article, true topic: sports
>
> `Steve Nash Announces Retirement From The NBA Nash played 19 years in the league.`

Topic	steve	nash	retirement	nba	league	7 more	z_k	$p(k \mid \text{text})$
international	−12.94	−12.94	−12.25	−12.25	−12.25	−51.95	**−129.30**	**0.000**
national politics	−9.99	−12.93	−10.73	−11.83	−11.54	−52.37	**−124.10**	**0.000**
business and economy	−10.59	−12.29	−8.89	−12.29	−11.19	−51.25	**−120.57**	**0.003**
entertainment	−9.88	−11.83	−11.83	−10.85	−10.44	−49.27	**−117.69**	**0.045**
sports	−10.42	−12.91	−10.60	−7.74	−8.03	−50.22	**−114.65**	**0.952**
science and technology	−10.11	−11.50	−12.88	−12.88	−12.88	−51.67	**−125.29**	**0.000**

Softmax turns the six scores into probabilities. Naive Bayes predicts sports, the true topic.

A misclassified article

> A dev article, true topic: business and economy
>
> `Markets Tumble Amid Brexit Chaos The vote to leave the EU has sparked a panic among investors around the world.`



Word	in international	in business
brexit	56	4
vote	73	15
eu	52	12
investors	1	53
markets	7	37

Predicted **international**, true topic **business and economy** wrong

How good is this model?

Naive Bayes on the dev set

Dev macro-F1, higher is better



Dev articles classified wrongly, of 4,000



79% of dev articles get a largest probability above 0.99

It is also very sure of itself, right or wrong.

Probability and loss

Text	True topic	Probability of the true topic
`Star film wins`	entertainment	**0.700**
`Steve Nash Announces Retirement …`	sports	**0.952**
`Markets Tumble Amid Brexit Chaos …`	business and economy	**0.057**

A loss is a function of this probability: $L(p)$

1. Probability ✓ → **2. Loss** → 3. Adjustment

Part 3 constructs the loss.

Part 3: Constructing the loss

One number that says how wrong the model was.

probability of the true topic → loss

Loss in classification

In classification, the model gives a probability to **every topic**.

`Star film wins`

sports	0.30	
entertainment	0.70	true topic

$$p = 0.70$$

The loss depends only on the probability of the true topic.

A loss $L(p)$ is a function of the probability p that must satisfy the following:

1. $L(1) = 0$
   No loss when the true topic gets probability 1.
2. L falls as p rises
   A larger probability gives a smaller loss.
3. L rises as p falls
   A smaller probability gives a larger loss.

> **What function can we use for the loss?**
>
> The simplest choice is $L(p) = 1 - p$, **but it has several problems.**

Cannot tell a confident error from an uncertain one

	Model A	Model B
Text 1	0.6	0.99
Text 2	0.6	0.99
Text 3	0.6	0.99
Text 4	0.6	0.02
Correct	4 of 4	3 of 4

Total loss with $1 - p$

Model A $0.4 + 0.4 + 0.4 + 0.4 = 1.60$



Model B $0.01 + 0.01 + 0.01 + 0.98 = 1.01$



Problem: with $1 - p$, Model B has a lower loss than Model A. It should be the opposite, because Model A is correct on every text.

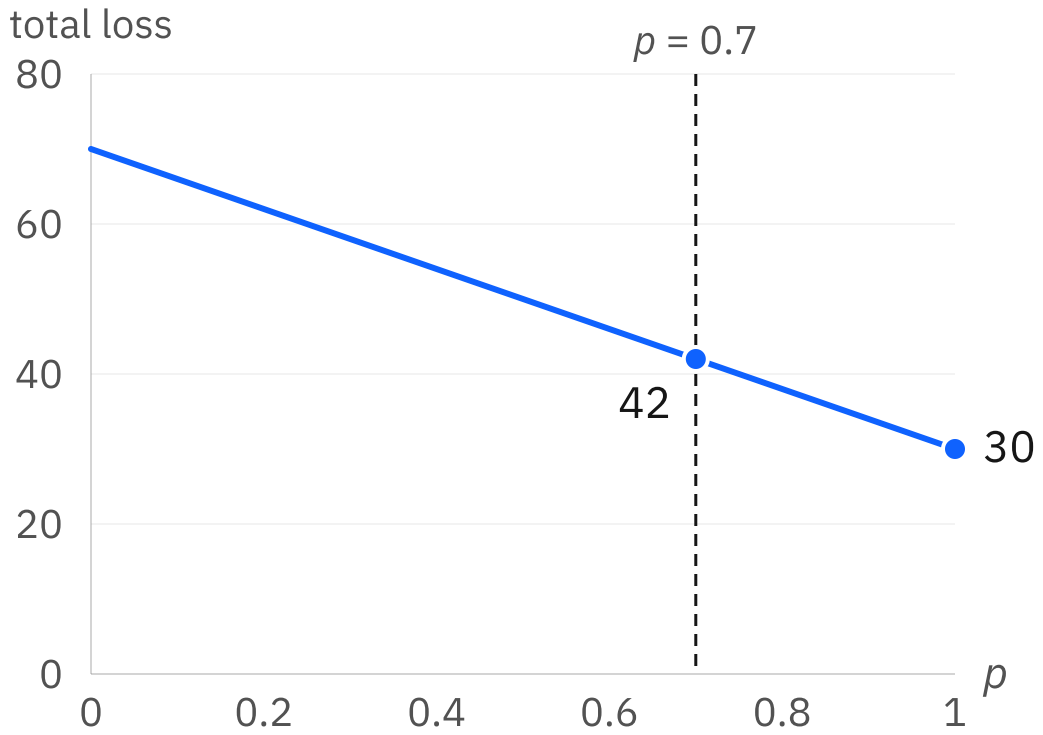
Prefers an overconfident answer to an honest one

Headlines	$p(\text{true topic})$	Loss
70 sports	p	$L(p) = 1 - p$
30 entertainment	$1 - p$	$L(1-p) = p$

$$\begin{aligned}\text{Total loss} &= 70\,L(p) + 30\,L(1-p) \\ &= 70\,(1-p) + 30\,p\end{aligned}$$

At $p = 0.7$ total loss $= 42$

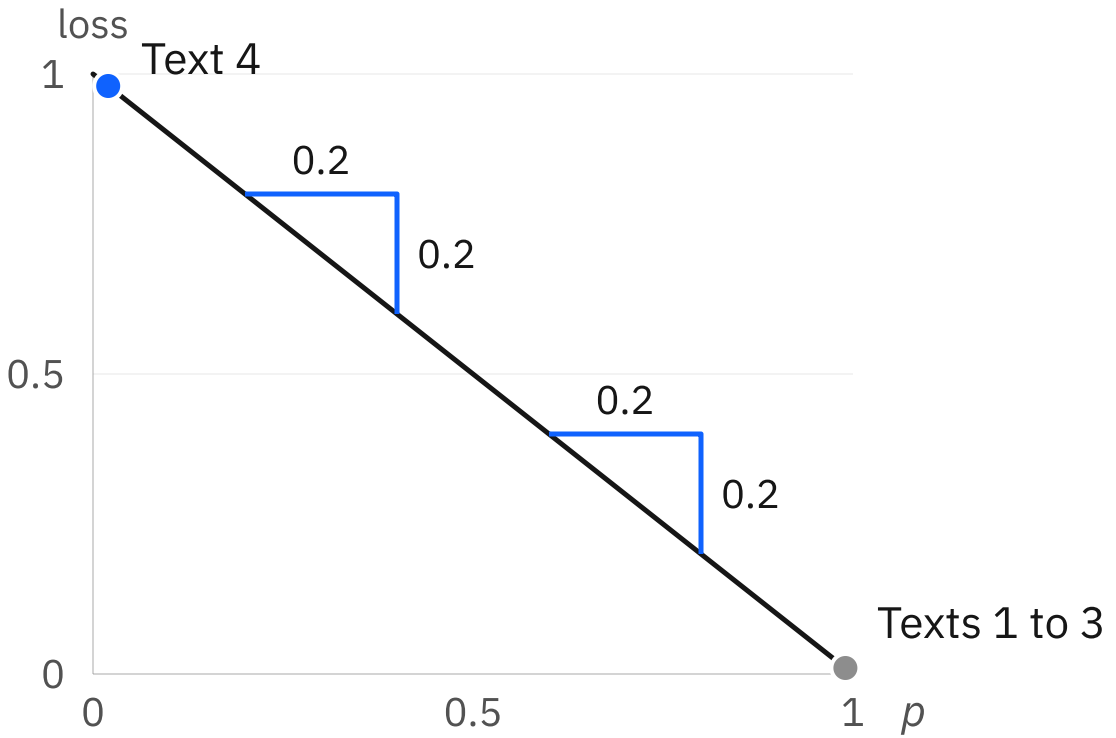
At $p = 1$ total loss $= 30$



Problem: with $1 - p$, the loss is lowest at $p = 1$, not at the honest answer $p = 0.7$. Training would make the model claim that every headline is sports.

A constant gradient slows learning from errors

	p	Loss $1-p$	Pull
Text 1	0.99	0.01	1
Text 2	0.99	0.01	1
Text 3	0.99	0.01	1
Text 4	0.02	0.98	1



Problem: with $1-p$, the mistake on Text 4 pulls no harder than a text that is already right.

Not additive over independent texts

Text 1: $p = 0.5$

Text 2: $p = 0.5$

One text at a time

$$(1 - 0.5) + (1 - 0.5) = 1$$

Both texts as one pair

$$1 - 0.5 \times 0.5 = 0.75$$

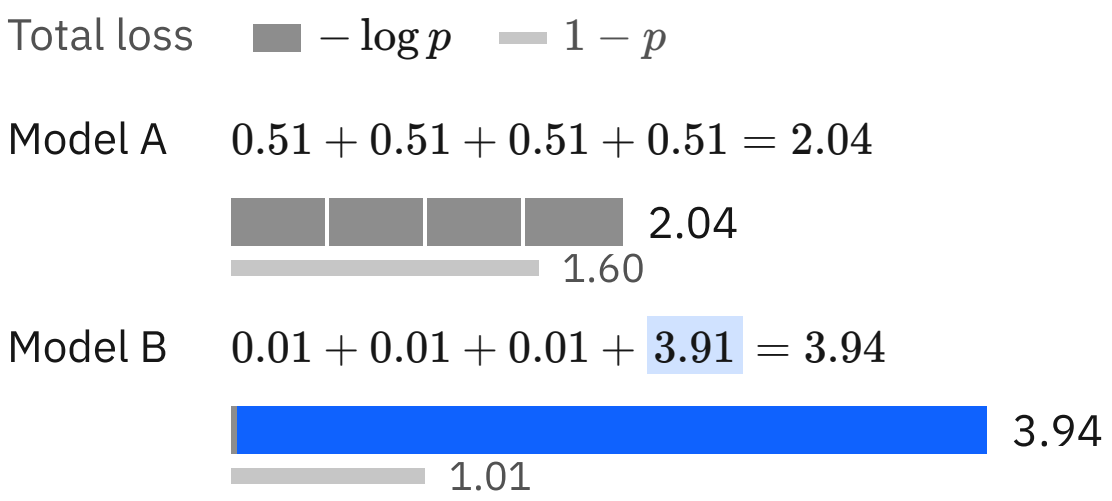
Texts	One at a time	As one group
2	1	0.750
4	2	0.938
8	4	0.996

Problem: with $1 - p$, the loss of the same texts depends on how we group them. A good loss gives the same answer either way.

A better loss function

$$L(p) = -\log p$$ log is the natural logarithm.

	Model A	Model B
Text 1	0.6	0.99
Text 2	0.6	0.99
Text 3	0.6	0.99
Text 4	0.6	0.02
Correct	4 of 4	3 of 4



Solved: $-\log p$ tells a confident error from an uncertain one. Model A, correct on every text, has the lower loss.

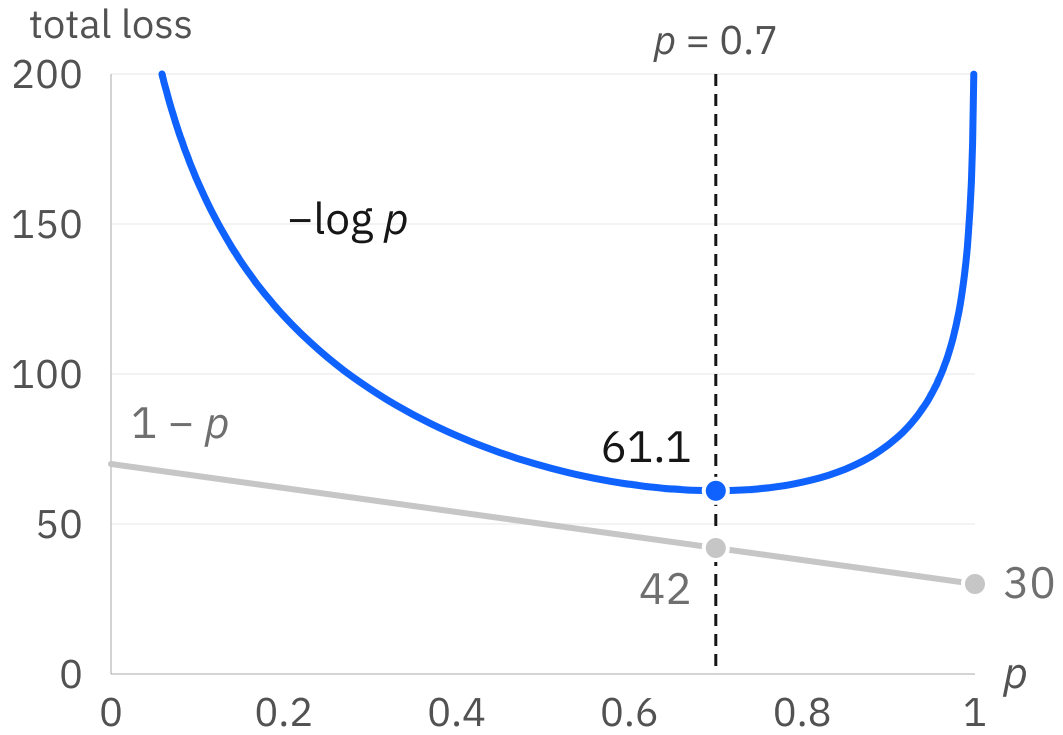
Prefers the honest answer

Headlines	$p(\text{true topic})$	Loss
■ 70 sports	p	$L(p) = -\log p$
■ 30 entertainment	$1 - p$	$-\log(1-p)$

$$\begin{aligned}\text{Total loss} &= 70\,L(p) + 30\,L(1-p) \\ &= -70\log p - 30\log(1-p)\end{aligned}$$

At $p = 0.7$ total loss $= 61.1$

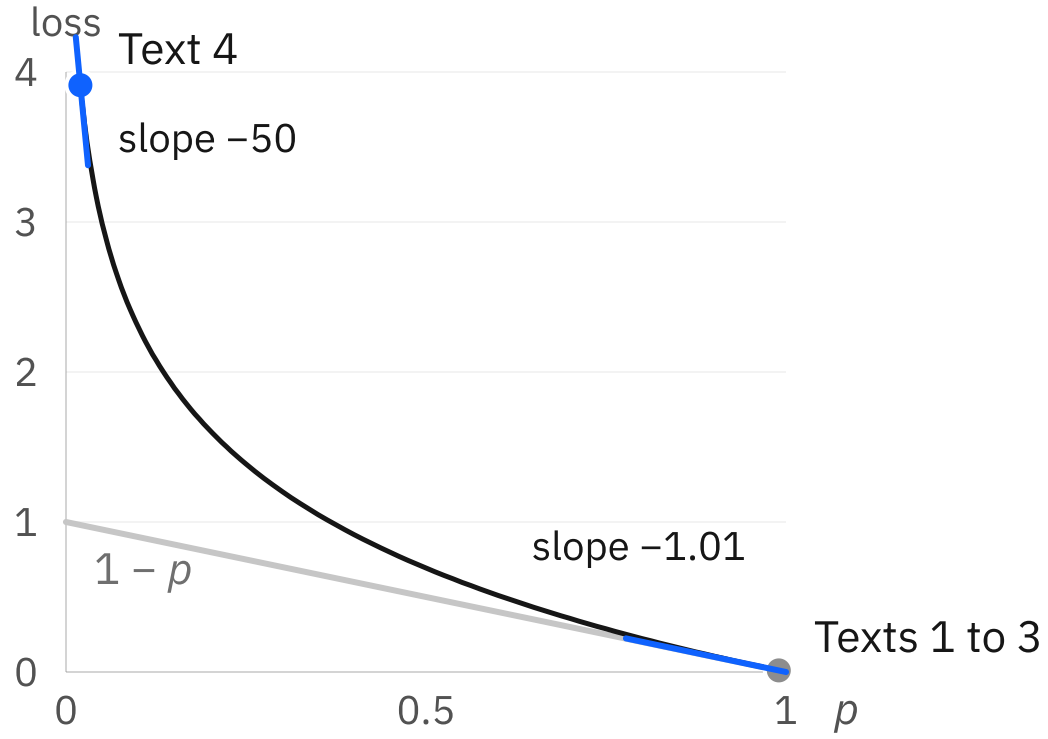
At $p = 1$ total loss $= \infty$



Solved: with $-\log p$, the total loss is lowest at the honest answer, $p = 0.7$.

A steeper gradient for larger errors

	p	Loss $-\log p$	Pull
Text 1	0.99	0.01	1.01
Text 2	0.99	0.01	1.01
Text 3	0.99	0.01	1.01
Text 4	0.02	3.91	50



Solved: with $-\log p$, the larger the error, the harder the text pulls.

Additive over independent texts

Text 1: $p = 0.5$ | Text 2: $p = 0.5$

One text at a time

$$-\log 0.5 - \log 0.5 = 1.39$$

Both texts as one pair

$$-\log(0.5 \times 0.5) = -\log 0.25 = 1.39$$

	$-\log p$		$1 - p$	
Texts	**Alone**	**Group**	**Alone**	**Group**
2	1.39	1.39	1	0.750
4	2.77	2.77	2	0.938
8	5.55	5.55	4	0.996

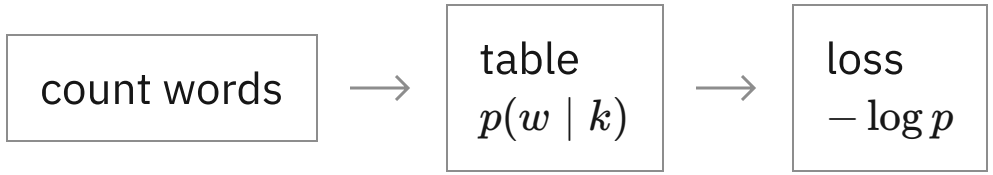
Solved: with $-\log p$, the losses of independent texts add up, however we group them.

What we learned

Bayes' formula gives the probability of a topic from the prior and the likelihood.	$p(k \mid \text{text}) = \dfrac{p(\text{text} \mid k)\, p(k)}{\sum_j p(\text{text} \mid j)\, p(j)}$
A text is a bag of words. The naive assumption multiplies the probabilities of its words.	$p(\text{text} \mid k) = \prod_i p(w_i \mid k)$
Logs prevent underflow, and softmax turns the scores back into probabilities.	$z_k = \log p(k) + \sum_i \log p(w_i \mid k)$ $p(k \mid \text{text}) = \text{softmax}(z)_k$
The loss $L(p) = -\log p$ is known as log loss, negative log-likelihood and, above all, **cross-entropy**.	$H(q, p) = -\sum_k q_k \log p_k = -\log p_{\text{true}}$
Naive Bayes is also called a unigram language model: each row of its table gives one word at a time.	$p(w \mid k)$

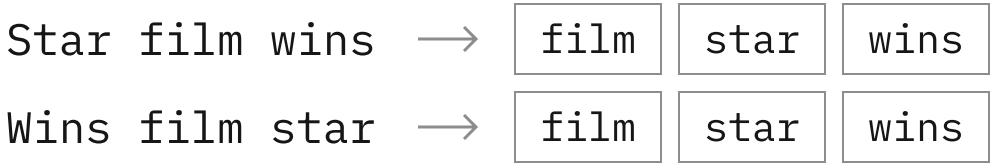
Limits of Naive Bayes

1. It cannot learn from its loss



no way back: the table has no weights for gradient descent to adjust

2. Word order is missing



the same bag

Class 3 An n-gram language model with trainable weights addresses both points.

3. But it can write

Naive Bayes can do something interesting: it can generate language.

Writing with the table

The sports row of the toy table, as a bar of length 1



Text written from the sports row

`match` `wins` `star` `match` `wins` `song`

Repeat the draw for every word. The table that classifies can also write.

Six topics write

international	national politics	business and economy	entertainment	sports	science and technology
on ল্যাভরভ নেতা ম্যালেরিয়ার দেশটির উদ্বিগ্ন গণমাধ্যম তবে র এবং শিশুর থিয়েটারে মানুষকে কী পরিকল্পনা	অন্যতম ঘটনা repeal front biden পুলিশের কয়েকদিন সউদী মোয়াজ্জেম আহমেদ হাওর তার বিচারকের এ party	s that ব্রাহ্মণবাড়িয়ার donald জামদানি প্রধান marked business since দেওয়া গতকাল in ই নতুন আয়	দিনের হবে নিয়েও ইকবাল of অভিনেতা খুব শিষ্য বাস্তবতা with সর্বশেষ করে পড়ে had got	চলে হারায় so পেসার টেস্টে brush who সেই ভারতে all why after ম্যাচের his corruption	of ourselves তোলারও if and মেগাপিক্সেল greenhouse কম্প্রোসার কিবোর্ড দিনের খাবার সেবা বাংলা ডাউনলোড দিচ্ছে

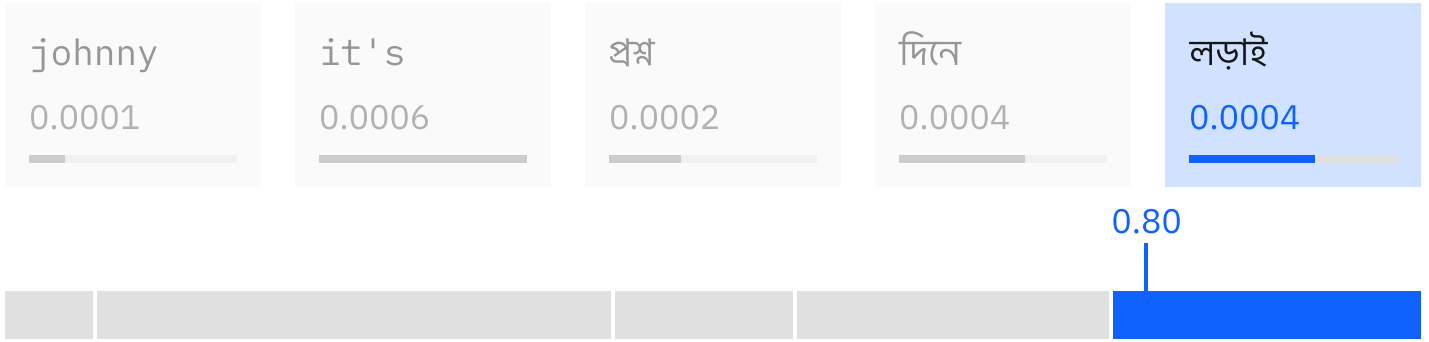
Each row writes in its own way, but without word order, and Bangla and English words mix.

Sampling from a Naive Bayes

Topic

- business and economy
- entertainment
- international
- national politics
- science and technology
- sports

One draw from the row



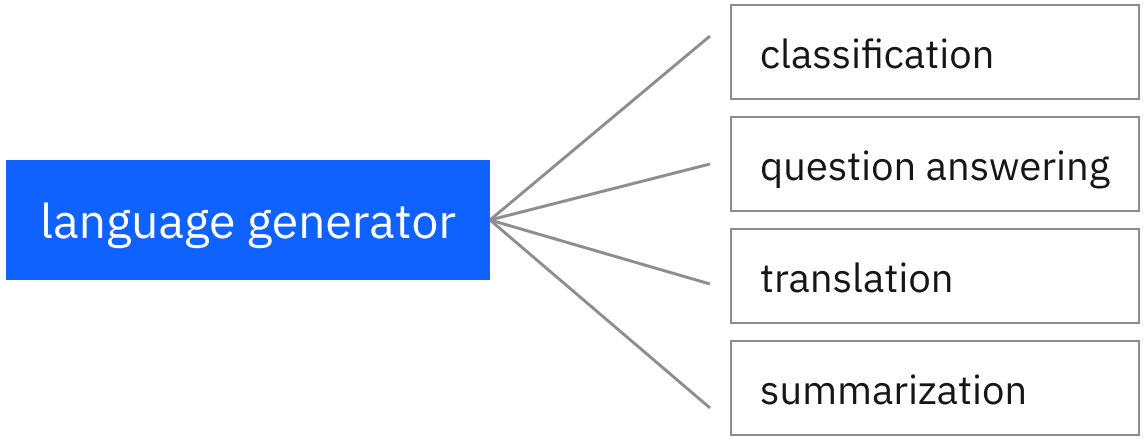
Document

sports — 15 of 15 words

3 and করে an কাটিয়ে i'm যাত্রা বাড়িয়ে she simone নেমে bay national ঠিকানা লড়াই

A language generator

The unigram model is not only a classifier. It is a language generator.



A language generator can be adapted to many tasks.

Our table

$p(w \mid \text{sports})$

Counted from 40,000 articles, one word at a time, without word order.

An LLM such as ChatGPT or Claude

$p(\text{next token} \mid \text{the words so far})$

Computed by a network from the context and word order, learned from an enormous amount of text in many languages.

Questions for class 3

1. We need a good generator.

2. How do we measure it?

   With its loss: cross-entropy, and from it perplexity.

3. How do we train it?

   With weights that gradient descent adjusts to lower the loss.

References

Textbook

Jurafsky, D. and Martin, J. H. (2026). *Speech and Language Processing*, 3rd ed. draft, August 2026. Chapter 4, sections 4.1, 4.3, 4.5 and 4.7–4.9, pp. 93–115 (Naive Bayes, logistic regression, cross-entropy, training); chapter 1, section 1.2, p. 9 (classification as language modelling); chapter 3, pp. 68–88 (reading for class 3).

Data and code

Class 1 notebook c01_minimal.ipynb (tokenizer, bag of words, nearest topic average) and the script c02_p2_build.py (Naive Bayes and the six topic texts), on the C1 training set (40,000 texts) and dev set (4,000 texts).

Papers

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D. and Sutskever, I. (2019). *Language models are unsupervised multitask learners.* OpenAI. (GPT-2.)

Thank you.